Table des matières

Cours 01 — Analyser les sources 1
Objectifs 1
1. Le métier 2
2. L’architecture de référence 2
3. Le cold path et le hot path 3
4. Le périmètre de la formation 4
5. Le cas d’étude : Projet Boréal 4
6. DuckDB 6
7. Le type deviné 6
8. Le premier regard, en pratique 6
À retenir 7
Exercices 8
Annexe A — Le détail des étapes du labo 8

Cours 01 — Analyser les sources

Théorie · Cours 1/12 — Formation Data Engineering · débutant

Où vous êtes : **source** → ingestion → lac → entrepôt → restitution.

Objectifs

- Situer l’ingénieur de données (*data engineer*) : ce qu’il livre, et à qui.
- Nommer les cinq étapes de l’architecture de référence.
- Distinguer le cold path du hot path.
- Connaître Boréal et ses six sources.
- Interroger des fichiers sur place avec DuckDB.

© 2026 iAlexMG. Tous droits réservés.

- Reconnaître un type mal deviné.
- Faire le premier regard d'une source : documentation, livraison, profilage.

---

1. Le métier

L'ingénieur de données amène la donnée de la source jusqu'à ceux qui s'en servent. Il fait quatre promesses :

Promesse	**Ce qui la casse**
Elle arrive, complète	une coupure, un fichier manquant, une erreur avalée en silence
Elle est rangée, dans un format ouvert	un dossier fourre-tout, un format propriétaire
Elle est à l'heure, et datée	un traitement qui glisse, une panne que personne ne voit
Elle est bonne	des doublons, des valeurs impossibles, un capteur figé

Un traitement doit pouvoir se rejouer sans doubler ni perdre une ligne. C'est l'idempotence, au cœur de la formation à partir du Cours 03.

Ses clients sont les autres métiers de la donnée, rarement le décideur :

Client	**Ce qu'il attend**
l'ingénieur analytique	la couche `RAW`, complète et à jour
l'analyste BI et les opérations	des tableaux de bord justes, certains en direct
le scientifique de données	des séries longues et propres
l'ingénieur ML	les mêmes données à l'entraînement et en production
l'ingénieur IA	une documentation complète, dont on connaît la version

2. L'architecture de référence

source → ingestion → lac → entrepôt → restitution

© 2026 iAlexMG. Tous droits réservés.

Étape	Rôle	Outils de la formation	Cours
Source	produit la donnée ; on ne la contrôle pas	ERP, historien, GMAO, garanties, télémétrie, API météo	03, 04, 08, 09
Ingestion	aller chercher la donnée sans la perdre ni la doubler	Python, Airflow, MQTT, Kafka	03, 04, 06, 09
Lac	garder le brut à bas coût, en formats ouverts	Parquet, Iceberg	02, 10
Entrepôt	servir la donnée en SQL	DuckDB	03, 08, 12
Restitution	la montrer à ceux qui décident	Grafana	12

- Lac (*data lake*) : des fichiers, peu coûteux ; la structure se décide à la lecture.
- Entrepôt (*data warehouse*) : des tables au schéma fixé d'avance, avec types et droits.
- Lakehouse : le lac, avec les avantages de l'entrepôt. Les données restent des fichiers ouverts (Parquet), mais un format de table (Iceberg, Delta Lake) les présente comme de vraies tables : un schéma, des mises à jour sûres, l'historique des versions. Le coût du lac, le confort de l'entrepôt (Cours 10).

DuckDB est le moteur SQL de référence, sur votre poste. Snowflake et Databricks ont une annexe aux Cours 03, 07 et 10.

3. Le cold path et le hot path

Le cold path traite la donnée par lots, le hot path en continu.

	Cold path, par lots	Hot path, en continu
Délai	minutes à heures	secondes
Exemple chez Boréal	les véhicules assemblés hier	le compresseur 2 vibre-t-il trop en ce moment ?
Priorité	complétude, exactitude, rejeu	délai, disponibilité
Coût	faible : on calcule, puis tout s'éteint	élevé : des services allumés jour et nuit
Outils	Airflow, Parquet, DuckDB, Spark	MQTT, Kafka, base de séries temporelles, Grafana
Cours	03 à 07, 10	08, 09

Le cold path fait référence : c'est lui qu'on corrige et qu'on rejoue. Le hot path sert la réaction immédiate.

© 2026 iAlexMG. Tous droits réservés.

À retenir — la plupart des besoins « temps réel » sont horaires. Avant de bâtir un hot path, demandez quelle décision se prend dans la minute qui suit la mesure.

4. Le périmètre de la formation

source → **Data Engineering → couche `RAW`** → Analytics Engineering → couche `MARTS` → Business Intelligence ou Data Science → Machine Learning Engineering

- La formation produit la couche `RAW` : le brut tel que reçu, daté, avec son origine.
- Elle ne renomme pas, ne décode pas, ne calcule aucun indicateur. C'est le travail de l'ingénieur analytique, à partir du `STAGING`.
- Une exception : les séries de l'usine (historien, compteur électrique). Seul l'ingénieur de données connaît assez la source pour les nettoyer. Il les livre dans le schéma `argent` (Cours 08).
- Python et Linux sont supposés acquis, et le SQL des Cours 01 et 02 de la formation SQL avancé : jointures, filtres, types, dates, agrégats. Le reste du langage, la formation le résume à son premier emploi et nomme le cours de SQL avancé qui l'enseigne. Snowflake a sa propre formation.

Ce que la formation livre au Cours 12 :

Produit	Qui le consomme	Cours
La couche `RAW`	l'ingénieur analytique	03, 12
Les séries de l'usine, nettoyées et ramenées à l'heure	l'ingénieur analytique, le scientifique de données	08
Le lac bronze et argent	l'architecte	02, 10
La base de séries temporelles	l'analyste BI	08, 09, 12
Les contrats de données	l'ingénieur analytique, l'architecte	11

Piège — nettoyer « un peu » au chargement. Une correction faite à l'ingestion n'est ni versionnée ni testée, et si elle se trompe, le brut est perdu. On charge tel quel, on corrige plus loin.

5. Le cas d'étude : Projet Boréal

Véhicules Boréal inc. (fictif) fabrique des motoneiges, des motomarines et des véhicules hors-route. Environ 150 concessionnaires commandent à l'ERP. L'usine de Val-des-Aulnes, au Saguenay–Lac-Saint-Jean, compte trois lignes d'assemblage, une cabine de peinture, un banc d'essai et des utilités (dont l'air comprimé).

Le vocabulaire de l'usine :

© 2026 iAlexMG. Tous droits réservés.

Terme	Sens	Exemple
SCADA (*Supervisory Control And Data Acquisition*)	le système qui lit les capteurs et pilote les équipements ; il ne garde pas l'historique	—
Historien	la base qui archive les mesures pendant des années	—
Étiquette (*tag*)	le nom d'un point mesuré	`UTL.CMP2.VIBRATION`
Actif	l'équipement, identifiant à tiret	`UTL-CMP2`, le compresseur d'air 2
GMAO	la gestion de la maintenance : les bons de travail	—

Les six sources :

Source	Ce que vous recevez	Ce qui la rend difficile	Cours
ERP (type SAP)	un extrait CSV par mois et par table ; une copie de la base	noms en abréviations allemandes, dates en texte, un extrait livré deux fois	03, 04
Usine	historien (un CSV par jour), actifs, GMAO, compteur électrique	points irréguliers, heure locale sans fuseau, trous, doublons	01, 02, 08
Garanties	un extrait par mois, réclamations en JSON	données personnelles, réclamation qui revient à chaque changement de statut	03
Télémétrie	une archive JSON quotidienne des sorties des véhicules	doublons, retards, format qui change	09, 10
Documentation	manuels, bulletins, politiques	deux versions d'une même politique	03
Météo	l'API d'Environnement et Changement climatique Canada	pagination, limites de débit, reprises	04

- Cinq sources sont générées, au Cours 00 (annexe A.0). La météo est réelle.
- Le numéro de série du véhicule (`MN26000417`) relie l'ERP, la télémétrie et les garanties.
- Les données portent deux cas d'usage des formations suivantes : la prévision de la demande électrique et la maintenance prédictive. Cette formation ne construit aucun modèle ; elle garantit des données complètes, datées et justes.

© 2026 iAlexMG. Tous droits réservés.

- Règle du terrain : jamais de pipeline branché sur le réseau qui commande les équipements. La donnée en sort par l'historien.

Pour aller plus loin : `_univers/boreal/README.md` (le décor), `_univers/boreal/DICTIONNAIRE.md` (chaque fichier et chaque colonne), `02_Boreal/Mise_en_situation_Boreal_01_DE.md` (le mandat).

6. DuckDB

- Un moteur SQL analytique, sans serveur : il tourne dans le programme qui l'appelle.
- Il interroge un fichier, ou tout un dossier par un motif (`*.csv`), sur place, sans rien charger.
- Il devine le type des colonnes d'un fichier qui n'en déclare pas.
- Il s'utilise en ligne de commande (`duckdb`), dans un programme Python, ou par son interface web (`duckdb -ui`), un carnet de requêtes semblable à une feuille Snowsight. Les labos prennent l'interface pour explorer, et la ligne de commande pour exécuter des scripts (`duckdb -f`, dès le Cours 02).

7. Le type deviné

Un CSV ne contient que du texte. Un JSON connaît les nombres et les chaînes, pas les dates. L'outil devine donc les types sur un échantillon. Il devine juste la plupart du temps, et la plupart de ses erreurs ne plantent pas :

Ce qui est deviné	**Ce qui casse**
une date `20251231` lue en entier	ajouter un jour donne `20251232`
une date `06/01/2022` laissée en texte	`MIN` et `MAX` trient des chaînes : `01/02/2024` passe avant `06/01/2022`
un code `01` lu en nombre	il devient `1`
un horodatage avec décalage (`-05:00`)	ramené à UTC, le décalage disparaît
une archive dont les premiers fichiers sont vides	tout devient une seule colonne, de type `JSON`
une clé lue en entier d'un côté (`7060400`), en texte de l'autre (`502S001`)	la jointure plante

Règle du Cours 02 : on déclare les types, on ne les devine pas.

8. Le premier regard, en pratique

Quand une source arrive, l'ingénieur de données la regarde avant d'écrire la moindre ligne de pipeline. C'est le profilage (*data profiling*). Il ne corrige rien : il mesure et il note. Les questions sont toujours les mêmes :

© 2026 iAlexMG. Tous droits réservés.

Question	Ce qu'on mesure	Chez Boréal (labo)
Que dit la source ?	la documentation livrée avec les fichiers	les sept `LISEZMOI.md` (2.1)
Tout est-il arrivé, intact ?	les fichiers, les octets, les fichiers vides, les empreintes	1 763 fichiers, 0 altéré (2.2 à 2.4)
Combien ?	les lignes de chaque table	275 436 lignes d'historien (3.1)
Sous quelle forme ?	les colonnes et leurs types	`DESCRIBE` (3.2)
Une ligne, c'est quoi ?	la clé, et si elle est unique	26 538 lignes pour 25 483 commandes (3.13)
Sur quelle période ?	le premier, le dernier, les trous	le compteur, complet au quart d'heure près (3.11)
Qu'est-ce qui manque ?	les valeurs vides	la précipitation horaire, toujours vide (3.24)
Quels codes ?	les codes et les drapeaux	qualité 192, 64, 0 (3.3) ; relevés estimés (3.11)
Des doublons ?	les lignes contre les distincts	2 303 messages en double (3.19)
La forme change-t-elle ?	les colonnes d'une période à l'autre	le micrologiciel 2.0 (3.20)
Les sources se tiennent-elles ?	les clés communes, les orphelins	le numéro de série, 0 orphelin (3.27)
Des données personnelles ?	les colonnes sensibles, la loi qui s'applique	un client effacé, encore nommé dans un ancien extrait (3.17)

Ce premier regard se fait une fois, à la main, quand une source arrive. Ce qu'il apprend devient ensuite des contrôles automatiques du pipeline : tests dbt, contrats de données (Cours 11). Les attendus du labo en sont l'ébauche.

À retenir

- Quatre promesses : la donnée arrive, rangée, à l'heure, bonne. Un traitement se rejoue sans rien casser.
- source → ingestion → lac → entrepôt → restitution.
- Le cold path fait référence ; le hot path coûte cher. La plupart des besoins « temps réel » sont horaires.
- La formation livre la `RAW` brute et les séries de l'usine nettoyées. Rien ne se corrige à l'ingestion.

© 2026 iAlexMG. Tous droits réservés.

- Boréal : six sources, une seule réelle, reliées par le numéro de série.
- DuckDB lit les fichiers sur place. Un type deviné se trompe en silence, ou fait planter une jointure.
- Le premier regard mesure sans corriger : la documentation, la livraison, puis le volume, la forme, la clé, la période, les vides, les codes, les doublons, les liens et les données personnelles de chaque source.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_01_DE.md`. Faites-les après le labo.

Annexe A — Le détail des étapes du labo

A.0 Le Cours 00 : le projet et ses sources

Notions : section 5, les six sources de Boréal.

- `mkdir Projet_Boreal` crée le projet. Le générateur ne le crée pas : `--projet` nomme un dossier qui doit exister, et il s'arrête avec un message clair s'il ne le trouve pas.
- Le générateur, `_univers/boreal/generer_boreal.py`, vit dans le dépôt des formations, avec les autres outils du parcours : on ne le copie nulle part. Il n'emploie que la bibliothèque standard de Python, et refait `sources/` de zéro à chaque lancement.
- Son tirage est fixé : tout le monde reçoit les mêmes octets. Il compare son empreinte globale à celle que versionne `_univers/boreal/REFERENCES.json`. « Conforme » veut dire que vos 1 763 fichiers sont ceux de l'auteur, octet pour octet ; « DIFFÉRENTE » donne les deux empreintes et renvoie à `sources/empreintes.sha256`, qui porte celle de chaque fichier.
- Le contrôle ne vaut que pour les générations que les cours citent : celle-ci, celle du Labo 06 (`--jusqua 2026-03`) et celle du Labo 07 (`--jusqua 2026-03 --historien complet`).
- On ne modifie jamais ces fichiers : ils simulent les exports d'autres systèmes. Ils ne vont dans aucun dépôt Git : 138 Mo de données brutes, qui se regénèrent à l'identique.

A.1 Étape 1 : le dépôt `01_de`

Labo : `Labo_01_DE.md`, étape 1.

© 2026 iAlexMG. Tous droits réservés.

Notions : aucune section de ce cours ; Git s'enseigne dans la formation Git et GitHub.

Dossier	Contenu	Versionné
`Projet_Boreal/`	la plateforme : sources, services, puis le lac et l'entrepôt	non : tout se reconstruit
`Projet_Boreal/01_de/`	votre code, qui deviendra le pipeline complet du Cours 12	oui, dépôt Git

- Ni données ni secrets dans Git. Les données sont hors du dépôt par construction.
- `.env` gardera le mot de passe de la base de l'ERP (Cours 04).
- `__pycache__/` reçoit les fichiers que Python compile.
- L'avertissement `LF will be replaced by CRLF` : Git normalise les fins de ligne sous Windows.

A.2 Étape 2 : la documentation et la livraison

Labo : `Labo_01_DE.md`, étape 2.

Notions : section 8, le premier regard ; section 5, les six sources de Boréal.

- 2.1. La documentation dit ce qu'aucune requête ne devine : l'heure locale sans fuseau de l'historien, les extraits qui répètent une commande modifiée, les messages MQTT livrés « au moins une fois », la Loi 25. On la lit avant la première requête ; chaque constat de l'étape 3 y renvoie.
- 2.2. `read_blob` lit le contenu et la taille de chaque fichier sans l'interpréter ; `**` descend dans tous les sous-dossiers. `replace` ramène les `\` de Windows à des `/`, et `split_part` garde le premier dossier sous `sources`. `COUNT(*) FILTER (WHERE size = 0)` ne compte que les lignes qui passent sa condition : l'agrégat filtré est au Cours 06 de la formation SQL avancé, section 6 ; ici, il suffit de le lire comme un `COUNT` qui a son propre `WHERE`. Les 41 fichiers vides de la télémétrie sont des jours sans message (son `LISEZMOI.md` le dit), pas une livraison ratée.
- 2.3. Le même inventaire, un niveau plus bas, pour les deux sources qui ont des sous-dossiers de nature différente.
- 2.4. Une empreinte SHA-256 résume le contenu d'un fichier en 64 caractères : un seul octet changé la change entièrement. `empreintes.sha256` donne celle de chaque fichier au départ ; `sha256(content)` recalcule celle du fichier reçu. Même nombre de fichiers et mêmes empreintes : rien ne manque et rien n'a été abîmé en route. En entreprise, un transfert de fichiers vérifie ainsi chaque livraison. La requête nomme d'abord deux résultats, `attendu` et `recu`, par `WITH`, puis les joint. Ces expressions de table communes sont au Cours 05 de la formation SQL avancé, sections 2 et 3 ; ici, chacune se lit comme une table provisoire, qui ne vit que le temps de la requête. `LEFT JOIN recu USING (fichier)` joint sur la colonne qui porte le même nom des deux côtés, et n'en garde qu'une (Cours 01 de SQL avancé, section 7).

A.3 Étape 3 : le profilage de chaque source

© 2026 iAlexMG. Tous droits réservés.

Labo : `Labo_01_DE.md`, étape 3.

Notions : section 8, le premier regard ; section 5, Boréal et son usine ; section 6, DuckDB.

L'usine : l'historien

- 3.1. Le motif `*.csv` lit les 92 fichiers de l'historien d'un coup, sans les copier.
- 3.2. DuckDB devine les types sur un échantillon. Ici, ils sont justes ; l'étape 4 montre qu'on ne peut pas s'y fier. L'interface montre aussi le type de chaque colonne au-dessus du résultat.
- 3.3. Les codes de qualité : 192 bonne, 64 douteuse, 0 mauvaise (valeur vide). Les 356 lignes à problème se perdent dans la masse (Cours 08 et 11).
- 3.4. Le SCADA lit chaque point aux deux minutes, mais l'historien n'en garde qu'une partie :
    - une mesure : un point quand la valeur sort de sa bande morte (le plus petit écart qui compte comme un changement), et au moins un par heure ;
    - un état ou un compteur : un point à chaque changement ;
    - toute étiquette : un point forcé à minuit, et une valeur de départ en tête de chaque fichier.

  D'où 27 661 points pour la température du banc d'essai, qui bouge sans arrêt, et 210 pour l'état de marche du compresseur 3, qui change rarement.
- 3.5. `UTL.CMP2.VIBRATION` le 1er octobre, dans l'ordre : la valeur de départ (la veille, 23 h 36), le point forcé de minuit, un point par heure jusqu'à 5 h (compresseur arrêté), puis des points rapprochés après le démarrage de 6 h. Les points ne tombent pas à intervalle régulier (Cours 08).
- 3.6. Un jour de production donne environ 3 600 lignes, un jour d'arrêt environ 1 800 : les fins de semaine, l'Action de grâce (13 octobre) et le congé des fêtes (24 au 31 décembre). Usine arrêtée, les valeurs bougent peu. Les plus gros fichiers (23, 24 et 27 octobre) tombent pendant la dérive du compresseur 2 : la pression du réseau d'air oscille, et ses étiquettes passent d'environ 356 points par jour à près de 1 000. Un fichier qui grossit peut signaler un équipement qui se dégrade.
- 3.7. 5 327 lignes sont en double. 5 278 sont les valeurs de départ : chaque fichier recopie en tête le dernier point de la veille, pour 58 étiquettes et 91 fichiers (le premier n'a pas de veille). Les 49 autres datent du 2 novembre : au retour à l'heure normale, l'heure de 1 h à 2 h se répète, et l'historien écrit en heure locale, sans fuseau. Le 275 436 de 3.1 est donc un nombre de lignes, pas de points de mesure. Le Cours 08 fait le ménage.

L'usine : le référentiel, la GMAO et le compteur

- 3.8. Le référentiel décrit chaque étiquette : son actif, son unité, sa bande morte, sa plage. Une étiquette absente du référentiel serait une mesure qu'on ne sait pas lire. Ici, aucune.
- 3.9. Une ligne, un bon de travail : `no_bt` est unique. Les 361 correctifs sont la matière de la maintenance prédictive (formation Data Science). Un bon encore ouvert n'a pas de date de fermeture : ses `ferme_le` vides sont normaux, pas des trous.

© 2026 iAlexMG. Tous droits réservés.

- 3.10. Les bons citent 17 des 23 actifs du référentiel, tous connus.
- 3.11. Un relevé aux 15 minutes, horodaté à la fin de l'intervalle : 1 461 jours × 96 = 140 256 quarts d'heure, moins le dernier, qui finit le 1er janvier 2026 à minuit et tombe dans un fichier pas encore livré. Avec 140 255 instants distincts, la série est complète. Les 239 relevés `E` ont été estimés après une perte de communication : on les garde, en sachant qu'ils sont estimés. Ici, DuckDB reconnaît le décalage (`TIMESTAMP WITH TIME ZONE`) : le type deviné est juste.

L'ERP

- 3.12. `union_by_name = true` lit les 9 tables d'un coup malgré leurs colonnes différentes ; `all_varchar = true` évite toute conversion. Les données maîtres (`KNA1`, `MARA`, `MAKT`, `T005U`) sont extraites en entier chaque mois : les 7 481 lignes de `KNA1` sont les mêmes concessionnaires, répétés dans 49 extraits. `CDHDR` et `CDPOS`, les documents de modification, n'existent que pour les 5 mois où une fiche de concessionnaire a changé.
- 3.13. Plus de lignes que de commandes : l'extrait d'un mois reprend chaque commande créée ou modifiée ce mois-là. Une commande modifiée plus tard revient dans un autre extrait.
- 3.14. `2023-03` et `2023-03_bis` contiennent les mêmes 609 commandes. Un pipeline qui charge tout ce qu'il trouve compterait mars 2023 deux fois, sans erreur (Cours 03). `UNION ALL` empile les deux comptes en un seul résultat ; les opérateurs d'ensembles sont au Cours 07 de la formation SQL avancé, section 6.
- 3.15. L'ERP ne vérifie aucune clé étrangère à l'extraction (son `LISEZMOI.md` le dit) : c'est à vous de le faire. Aucun poste de commande (`VBAP`) sans son en-tête, aucune commande d'un concessionnaire absent de `KNA1`.

Les garanties

- 3.16. Une ligne n'est pas un objet : un client revient à chaque mois où il change (25 379 lignes pour 23 527 clients), une réclamation à chaque changement de statut (5 623 pour 3 348). Pour l'état courant, on garde la version du mois le plus récent (Cours 03). Les enregistrements et les visites, eux, n'arrivent qu'une fois.
- 3.17. La Loi 25, la loi québécoise sur les renseignements personnels : un client qui demande l'effacement voit son nom, son courriel, son téléphone et son adresse vidés à partir de l'extrait du mois de sa demande. Les extraits déjà livrés les gardent : `CL0011163`, effacé en septembre 2025, est encore nommé dans l'extrait de janvier 2024. Un lac qui garde tous les extraits garde donc des données qu'il faudrait effacer. La requête affiche `nom IS NOT NULL` plutôt que le nom : on explore sans exposer.

La télémétrie

- 3.18. DuckDB lit le JSON lines (un objet JSON par ligne) comme un CSV.
- 3.19. `union_by_name = true` lit chaque fichier avec sa propre forme et réunit les colonnes par nom : c'est ce qui évite le piège de 4.6. Le courtier MQTT livre chaque message au moins une fois : 2 303 messages arrivent deux fois, avec le même `message_id` (Cours 09).

© 2026 iAlexMG. Tous droits réservés.

- 3.20. Le micrologiciel 2.0, diffusé à partir du 2025-05-01, renomme `temp_moteur_max` en `temperature_moteur_max` et ajoute `heures_moteur`. Une colonne qui change de nom coupe une série en deux, sans erreur, pour qui ne le sait pas (Cours 10).
- 3.21. Un véhicule hors de la couverture cellulaire publie ses messages à son retour : 291 arrivent plus d'un jour après la fin de leur sortie, jusqu'à 4 jours. Un traitement qui ne lit que le fichier du jour les manque (Cours 09).

La météo

- 3.22. La seule source réelle. 11 stations × 1 551 jours = 17 061 : chaque station a toutes ses journées. L'horaire de Bagotville compte 37 068 heures sur les 37 224 de la période : 156 manquent. La période dépasse celle de Boréal, jusqu'au 2026-03-31.
- 3.23. Une journée présente peut être vide : de 2 jours sans température moyenne à Bagotville à 67 à Charlottetown.
- 3.24. `PRECIP_AMOUNT` est toujours vide : sans valeur à analyser, DuckDB la devine en texte. `UTC_DATE` manque sur 24 heures. Attention aussi aux coordonnées de `stations.csv` : `452814000` veut dire 45° 28′ 14″, pas 452 millions de degrés.

Le corpus

- 3.25. Le catalogue (`index.csv`) et le disque se correspondent : 33 documents de chaque côté. Un document au catalogue sans fichier, ou l'inverse, serait le premier constat à signaler.
- 3.26. Deux politiques de garantie sont en vigueur en même temps : GAR-2024 remplace GAR-2022 pour les années-modèles 2025 et suivantes, pas pour les plus anciennes (Cours 03).

Les liens entre les sources

- 3.27. Le numéro de série relie les sources. `ANTI JOIN` garde les lignes de gauche sans correspondance à droite : aucune série de la télémétrie ni des garanties n'est inconnue de l'ERP. C'est une écriture de DuckDB ; l'anti-jointure, et le piège que cette écriture tend sur PostgreSQL, sont au Cours 07 de la formation SQL avancé, section 4. 6 759 véhicules publient sur 27 775 : seuls les véhicules connectés dont le propriétaire consent.
- 3.28. Le numéro de concessionnaire garde ses dix chiffres dans les deux systèmes : la jointure marche sans conversion.

En entreprise, les fichiers ne sont pas sur votre poste : ils arrivent dans un stockage infonuagique, comme Amazon S3 ou Azure Blob Storage. Le geste reste le même, seul l'outil change :

Geste du labo	**Snowflake (feuille Snowsight)**	**Databricks (notebook)**
Mesurer la taille des fichiers (2.2)	`LIST @stage/historien/`	`LIST 's3://…/historien/'`
Compter sans charger (3.1)	`SELECT COUNT(*) FROM @stage/historien/ (FILE_FORMAT => 'mon_format_csv')`	`SELECT COUNT(*) FROM read_files('s3://…/historien/', format => 'csv')`

© 2026 iAlexMG. Tous droits réservés.

Geste du labo	Snowflake (feuille Snowsight)	Databricks (notebook)
Voir les types devinés (3.2)	`SELECT * FROM TABLE(INFER_SCHEMA(LOCATION => '@stage/historien/', FILE_FORMAT => 'mon_format_csv'))`	`DESCRIBE QUERY SELECT * FROM read_files('s3://…/historien/')`

A.4 Étape 4 : les types devinés

Labo : `Labo_01_DE.md`, étape 4.

Notions : section 7, le type deviné ; section 6, DuckDB.

- 4.1. Ce que DuckDB devine pour `VBAK` :
  - `VBELN`, `KUNNR` : texte. Correct, les zéros de tête sont gardés.
  - `ERDAT`, `VDATU`, des dates SAP : entier. Faux, voir 4.2.
  - `AEDAT`, la même sorte de date : texte, parce qu'elle vaut souvent `00000000`. Deux colonnes de dates, deux types.
  - `VKORG`, `VTWEG`, des codes : entier. Un code n'est pas un nombre.
  - `NETWR`, un montant : `DOUBLE`, un nombre à virgule approximatif. Un montant veut un décimal exact.
- 4.2. Pour DuckDB, `ERDAT` est un nombre : 20251231 plus 1 donne 20251232, sans erreur.
- 4.3. `all_varchar = true` coupe la détection : tout arrive en texte, et c'est vous qui convertissez. La couche `RAW` gardera l'ERP ainsi (Cours 03).
- 4.4. Le message est arrivé à 0 h 25, heure de l'Est. DuckDB affiche 5 h 25 : il l'a ramené en UTC.
- 4.5. Lu en texte, le `-05:00` réapparaît. L'instant restait juste, mais le fuseau où roulait le véhicule est perdu.
- 4.6. Les 39 premiers jours de l'archive (du 2023-08-01 au 2023-09-08) sont des fichiers vides. La détection lit les premiers fichiers, n'y trouve rien, et rend une seule colonne de type `JSON`. Une requête sur `message.sernr` échouerait ensuite (`Binder Error: Referenced table "message" not found!`). La parade, `union_by_name = true`, sert depuis 3.19 ; le Labo 02 déclare en plus les colonnes.
- 4.7. `ouvert_le` suit le format de la GMAO, `JJ/MM/AAAA HH:MM`. DuckDB le laisse en texte, et `MIN` et `MAX` comparent alors des chaînes, caractère par caractère : `01/02/2024` passe avant `06/01/2022`, et les deux bornes tombent en 2024. Aucune erreur.
- 4.8. `STRPTIME` convertit avec le format connu : du 6 janvier 2022 au 29 décembre 2025.
- 4.9. `CLIMATE_IDENTIFIER` est un identifiant, pas un nombre. Dans l'horaire, il ne vaut que `7060400`, et DuckDB le devine entier ; dans `stations.csv`, `502S001` (Winnipeg) le force en texte. Pour joindre, DuckDB tente de convertir le texte en entier et plante. Ce piège-là, au moins, se voit.

© 2026 iAlexMG. Tous droits réservés.

- 4.10. `types = {...}` déclare le type d'une seule colonne et laisse deviner les autres.

A.5 Étape 5 : la sauvegarde

Labo : `Labo_01_DE.md`, étape 5.

Notions : section 6, DuckDB : l'interface pour explorer, la ligne de commande pour les scripts.

- Le carnet de l'interface vit dans `C:\Users\<vous>\.duckdb\extension_data\ui\` : ni versionné, ni partagé, ni sauvegardé avec le projet. Il sert à explorer ; ce qui mérite d'être gardé passe dans le dépôt, comme du code.
- `duckdb -f` rejoue un fichier d'un trait et s'arrête à la première erreur : c'est pourquoi 4.9, qui plante exprès, passe en commentaire. C'est le rôle de la ligne de commande dès le Cours 02.
- Le dossier `01_de\sql` recevra aussi les scripts du bronze, au Labo 02.

© 2026 iAlexMG. Tous droits réservés.